Unit 10 Processes

Course code LX13
Linux Basics

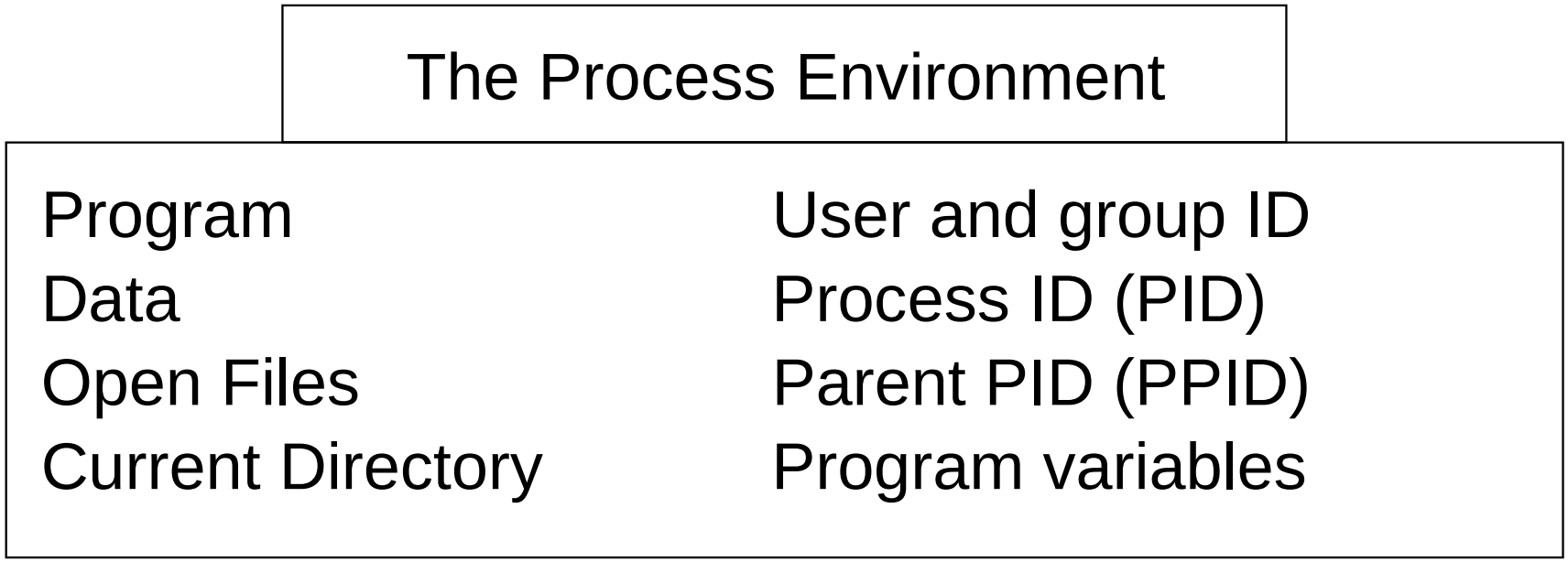
Objectives

After completing this unit, students should be able to:

- Define a Linux process
- Describe the relationship between parent and child processes
- Create and invoke shell scripts

What is a Process?

- A process is a task
- A running program is an example of a process



- To see the PID of the current shell enter:

```
$ echo $$
```

Login Process Environment

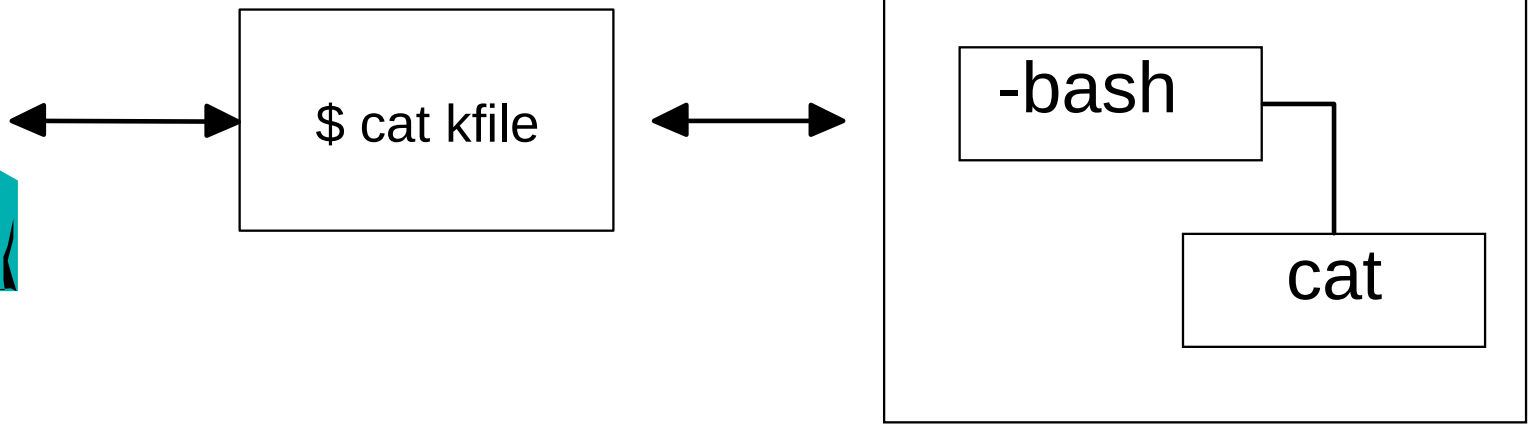
Login:

Linux System

-bash

Environment	
Program	-bash
uid	john
gid	john
file	/dev/tty1
PID	201

Process Environment



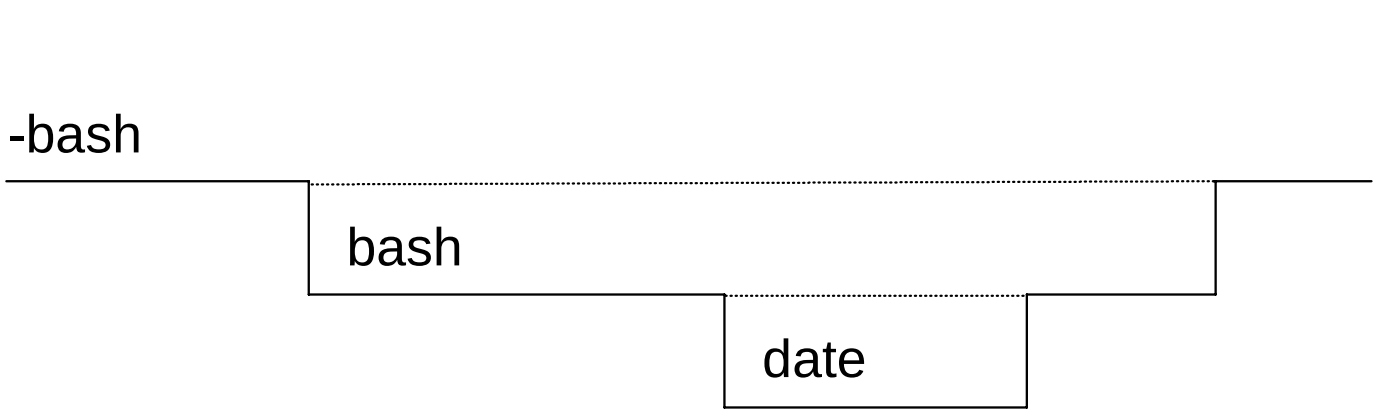
Environment during execution			
Program	cat	Parent	-bash
uid	john	PID	310
gid	john	PPID	201
file	/dev/tty1		
	kfile		

Parents and Children

```
$ echo $$
561

$ bash
$ echo $$
675

$ date
Thu Mar 25 22:28:21 CET 1999
$ <ctrl-d>
```



PID	PPID
561	1
675	561
777	675

Variables and Processes

- Variables are part of the environment of a process
- Processes cannot access or change variables in the environment of another process

```
$ x=4
$ bash
$ echo $x

$ x=1
$ echo $x
1
$ exit
$ echo $x
4
```

Exporting Shell Variables

- The **`export`** command is used to pass variables from a parent to a child process.
- Changes made to variables in a child process do not affect the variables in its parent.

```
$ export x=4
$ bash
$ echo $x
4
$ x=100
$ echo $x
100
$ exit
$ echo $x
4
```

What is a Shell Script?

- A shell script is a collection of commands stored in a text file.

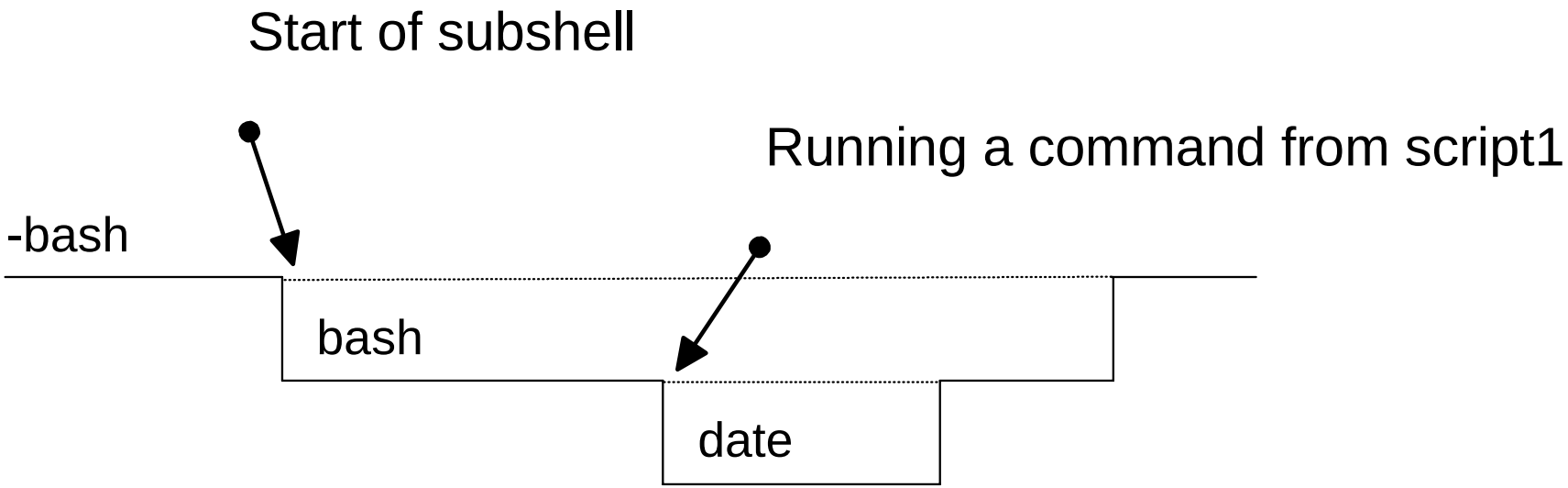
```
$ pwd
$ date
$ ls -l
```

```
$ cat script1
pwd
date
ls -l
$
```

Invoking Shell Scripts

- The script does not have to be marked executable but must be readable.
- bash invokes a script in a subshell

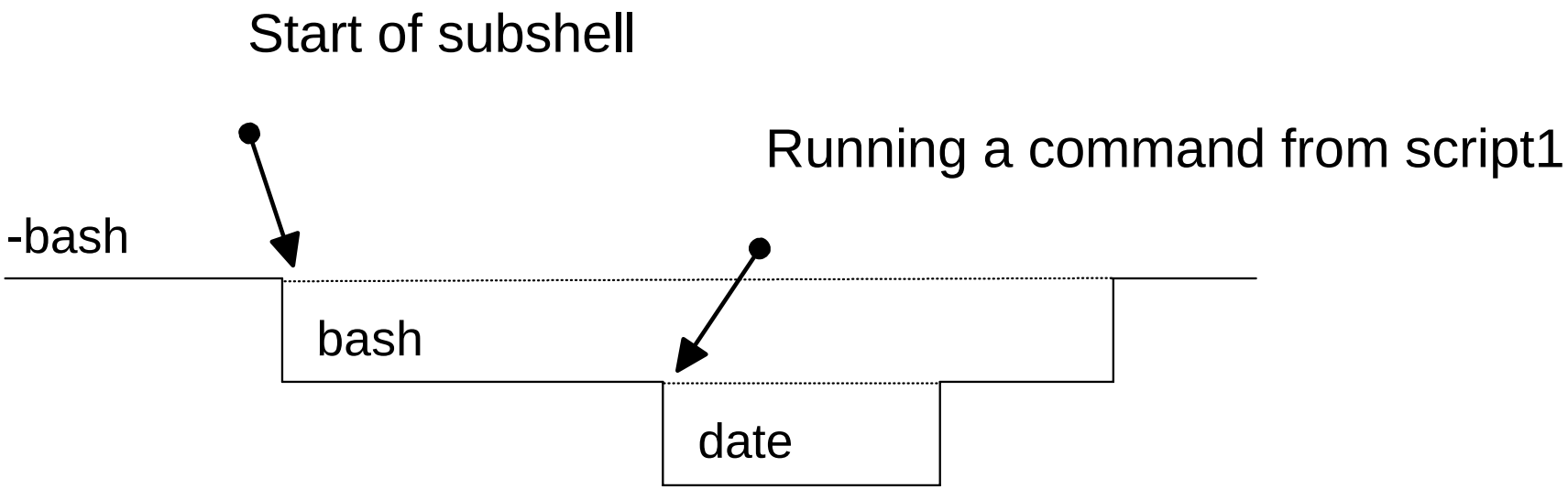
```
$ cat script1
date
$ bash script1
```



Invoking Shell Scripts *(2)*

- Use the `chmod` command to make the script executable.
- Then run the script as if it were a command
- The script is run in a subshell

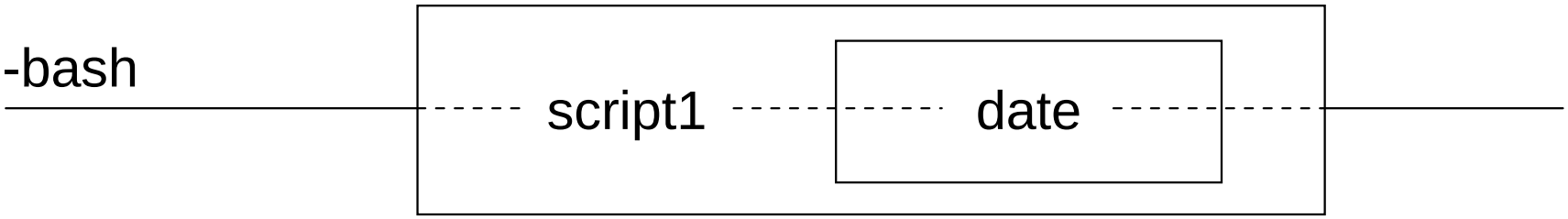
```
$ script1
```



Invoking Shell Scripts *(3)*

- Use the **.** (dot) command to execute the script in your current shell environment
- Scripts executed with the dot command can change your current environment

```
$ . script1
```



Return Codes from Commands

- A command returns a value to the parent process. By convention, zero means success and a non-zero value means an error occured.
- A pipeline returns a single value to its parent.
- The environment variable **$?** contains the return code of the previous command.

```
$ whoami
tux1
$ echo $?
0
```

Checkpoint

1. When would you execute a shell script using the dot (.) notation?

---

2. What is the command that is used to carry down the value of a variable into a subshell?

Checkpoint *(2)*

3. What would be the value of x at the end of the following steps?

```
$ ( ... login shell ... )
$ bash
$ x=50
$ export x
<ctrl-d>
$ echo $x
```

What is the value of x set to now?

Unit Summary

- Shells can be invoked in three ways:

```
$ bash scriptname      (must have read permission)
$ scriptname           (must have read and
                        execute permissions)
$ . scriptname         (must have read permission)
```

- A Linux process is a job, command or running program

- Every process has an environment in which it runs much of which is inherited from its initiating process, the parent process